

WSNs & IoT – Wireless Sensor Networks & Internet of Things

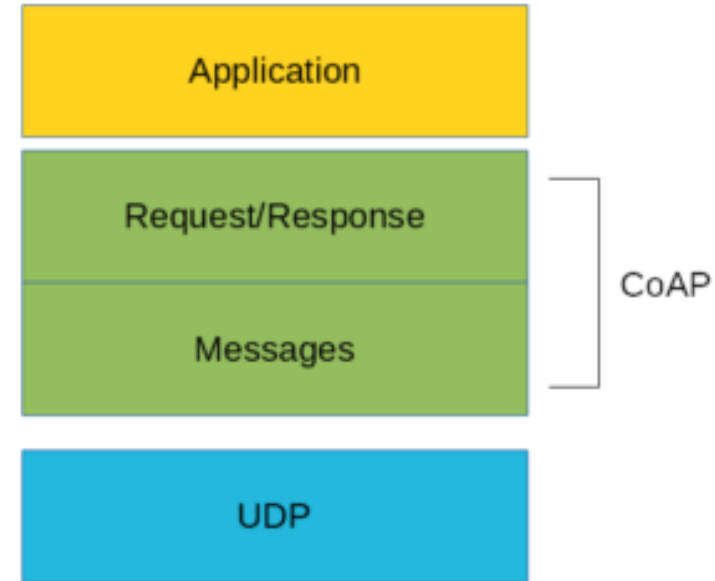
Tema 5 – Diseño de Redes

Protocolos de aplicación

Tema 5 WSNs & IoT

CoAP

- *Constrained Application Protocol*
- Protocolo M2M **muy ligero**
- Semántica **REST**
- Basado en **UDP**
- Intercambio de mensajes **asíncrono**
- Bajo *overhead*
- Fácil de analizar
- Compatibilidad con URI y Content-Type
- Capacidades de proxy y almacenamiento caché



CoAP Vs MQTT

- **CoAP**

- Solicitud / Respuesta
- Transporte UDP
- Basado en solicitudes

- **MQTT**

- Publicación / Suscripción
- Transporte TCP
- Impulsado por eventos

CoAP (II)

- **Capa de mensajes**

- Se ocupa de UDP y mensajes asincrónicos

- **Capa de solicitud/respuesta**

- Gestiona la interacción solicitud/respuesta

- **Tipos de mensajes**

- (Sol) **CONF**irmable
- (Sol) **NON**confirmable
- (Res) **ACK**nowledgement
- (Res) **RESET**

- **Términos**

- *Endpoint*: entidad que participa en el protocolo (normalmente, un host)
- Remitente: entidad que envía el mensaje
- Destinatario: entidad que recibe el mensaje
- Cliente: entidad que envía una solicitud y destino de la respuesta
- Servidor: entidad que recibe una solicitud y devuelve una respuesta

CoAP – Capa de mensajes

- Mensajes **binarios**

- [4B] **Fixed-size header**

- [2b] Version
- [2b] Type
 - (Req) CONfirmable
 - (Req) NONconfirmable
 - (Res) ACKnowledgement
 - (Res) RESET
- [4b] ToKeN Length
- [1B] Req/Res Code
 - [3b] Class
 - [5b] Code
- [2B] Message ID

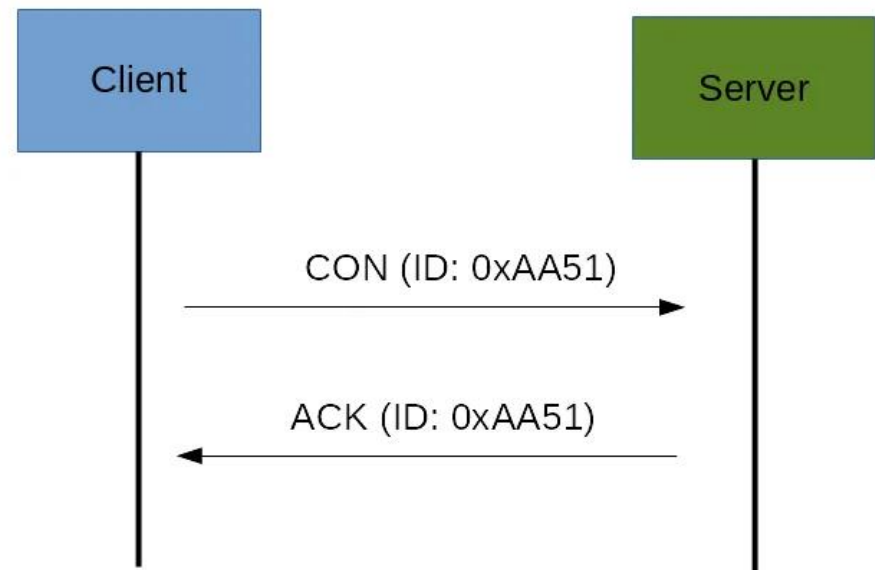
- [0 – 8B] **Token**

- **Options**

- Optimized type – length – value list

- **Payload**

- Preceded by 1B equal to 0xFF



CoAP – Capa de mensajes

Class.Code

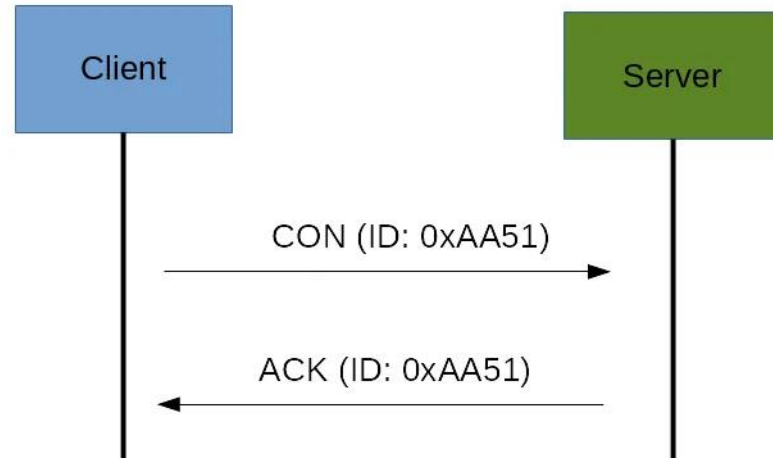
- Method: 0.XX
 - EMPTY
 - GET
 - POST
 - PUT
 - DELETE
 - FETCH
 - PATCH
 - iPATCH
- Success: 2.XX
 - Created
 - Deleted
 - Valid
 - Changed
 - Content
 - Continue
- Client Error: 4.XX
 - Bad Request
 - Unauthorized
 - Bad Option
 - Forbidden
 - Not Found
 - Method Not Allowed
 - Not Acceptable
 - Request Entity Incomplete
 - Conflict
 - Precondition Failed
 - Request Entity Too Large
 - Unsupported Content-Format
- Server error: 5.XX
 - Internal server error
 - Not implemented
 - Bad gateway
 - Service unavailable
 - Gateway timeout
 - Proxying not supported
- Signaling Codes: 7.XX
 - Unassigned
 - CSM
 - Ping
 - Pong
 - Release
 - Abort

CoAP – Capa de mensajes

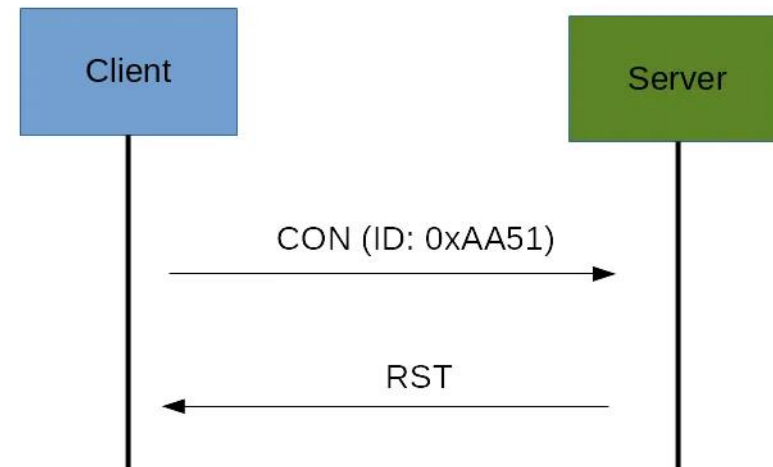
Mensaje CONfirmable

- Mensaje **confiable**

- Necesario ya que el transporte UDP no garantiza la entrega



- Si el servidor no puede procesar la solicitud, devuelve un mensaje RST



CoAP – Capa de mensajes

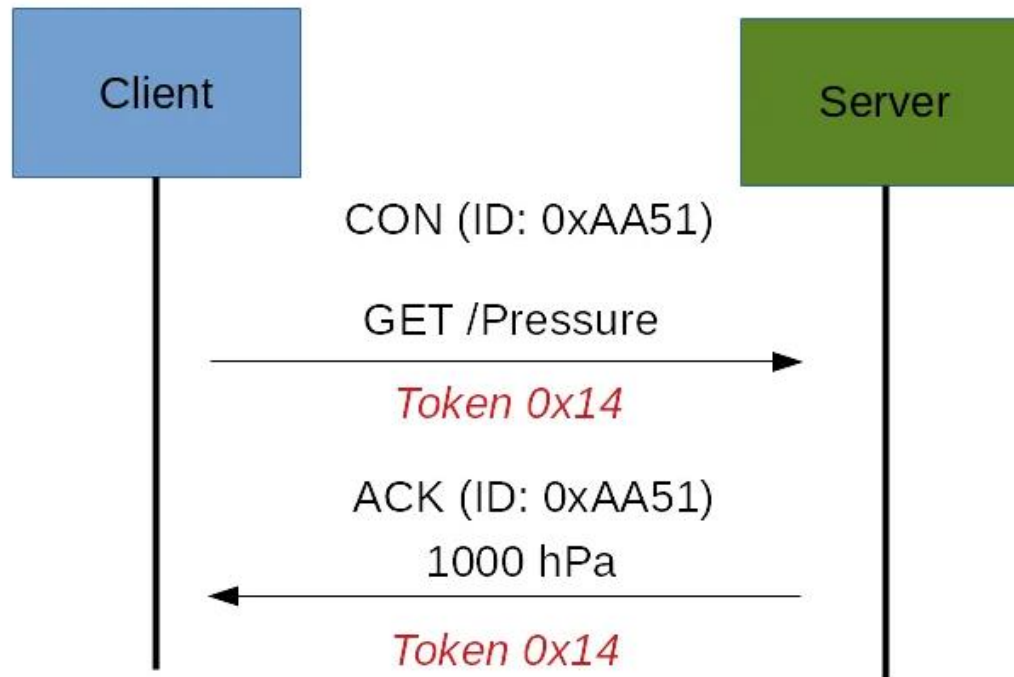
Mensaje NONconfirmable

- Mensaje **no confiable**
 - UDP no garantiza la entrega



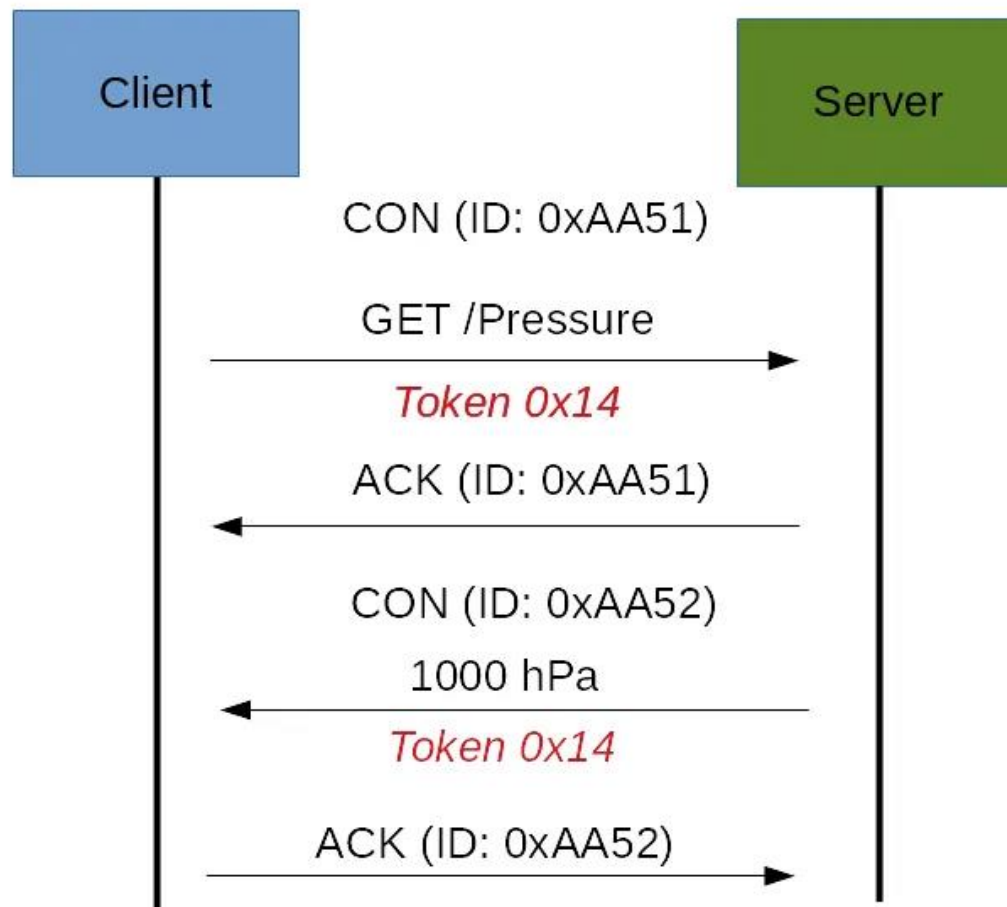
CoAP – Capa Sol/Res

- **Respuesta del mismo tipo** (CON/NON) que la **solicitud**
- Respuesta **síncrona**: el servidor puede responder de inmediato



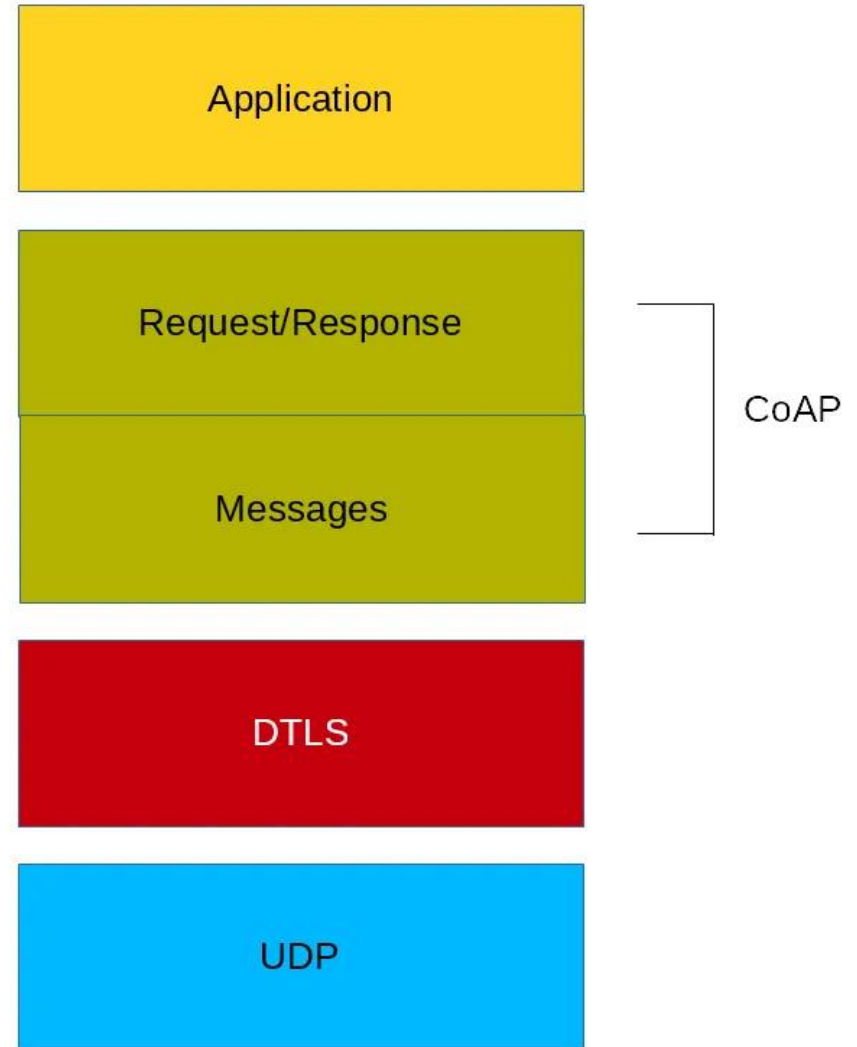
CoAP – Capa Sol/Res (II)

- Resp. **asíncrona**: servidor no puede responder inmediatamente
 - El **token** se utiliza para hacer **coincidir Sol y Res**



CoAP – Seguridad

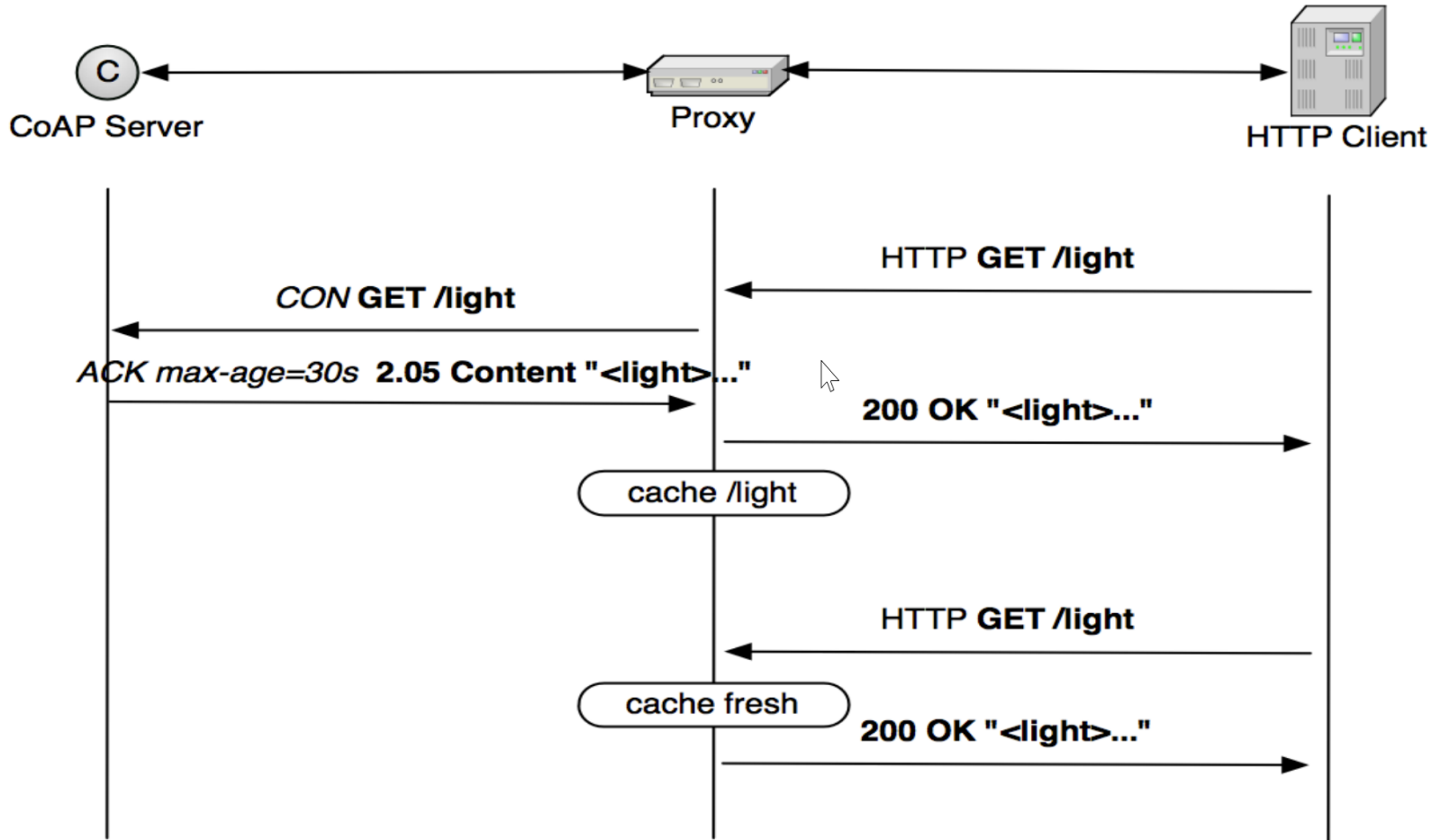
- **DTLS** (Datagram TLS)
 - Soporta RSA, AES, etc.
 - Algunos conjuntos de cifrado pueden no estar disponibles en dispositivos restringidos



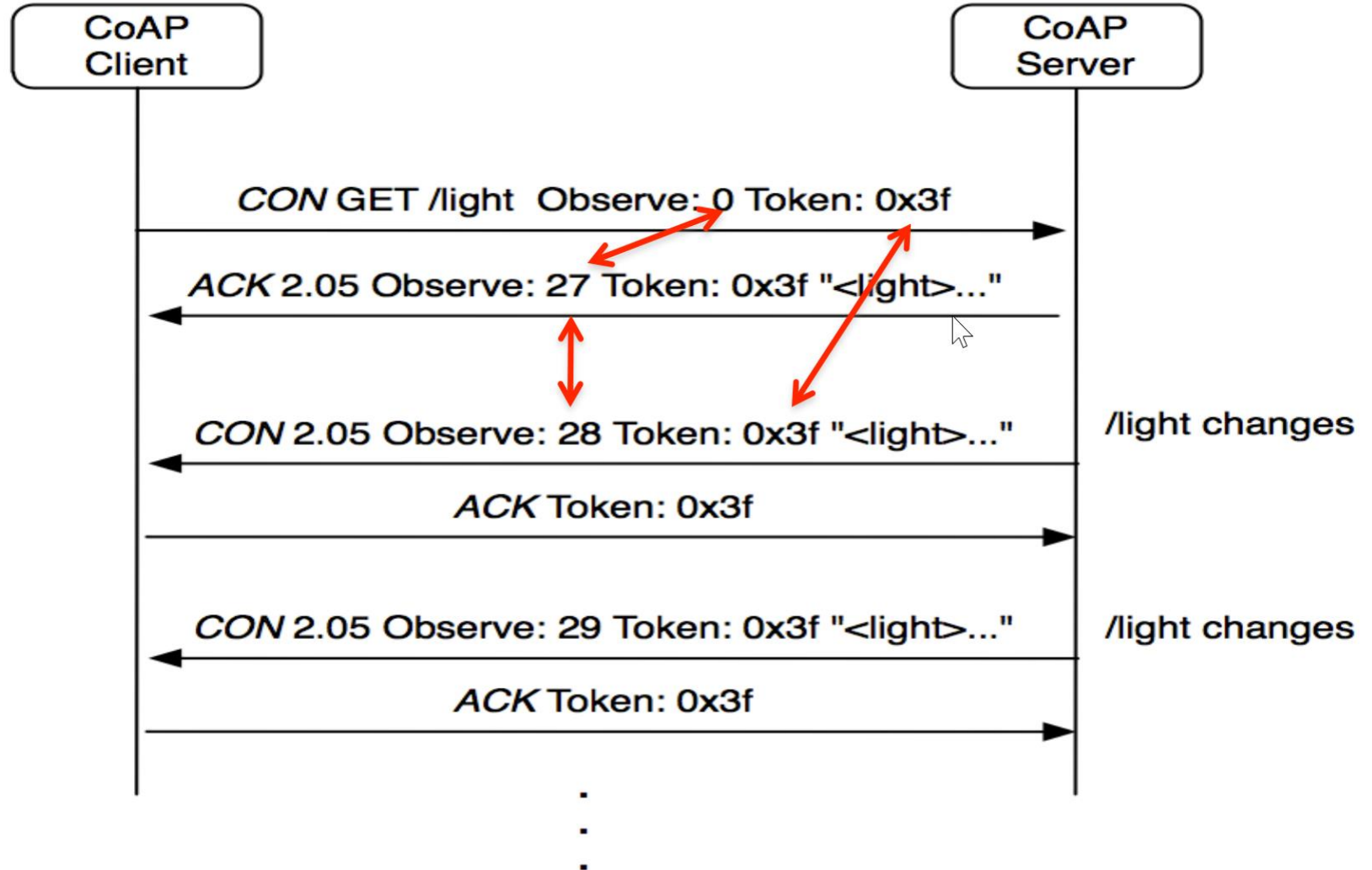
CoAP – Caching

- CoAP incluye un **modelo simple de *caching***
 - Capacidad de *caching* determinada por el código de respuesta
 - Una máscara de número de opción determina si se trata de una clave de caché
- Modelo de **frescura**
 - Opción **Max-Age** indica la duración de la caché
- Modelo de **validación**
 - Validez comprobada con la opción **Etag**
- El *caching* a menudo se soporta a través de un **proxy**
 - (En gral.) En nombre de un nodo restringido,
 - un nodo dormido,
 - o para reducir la carga de la red

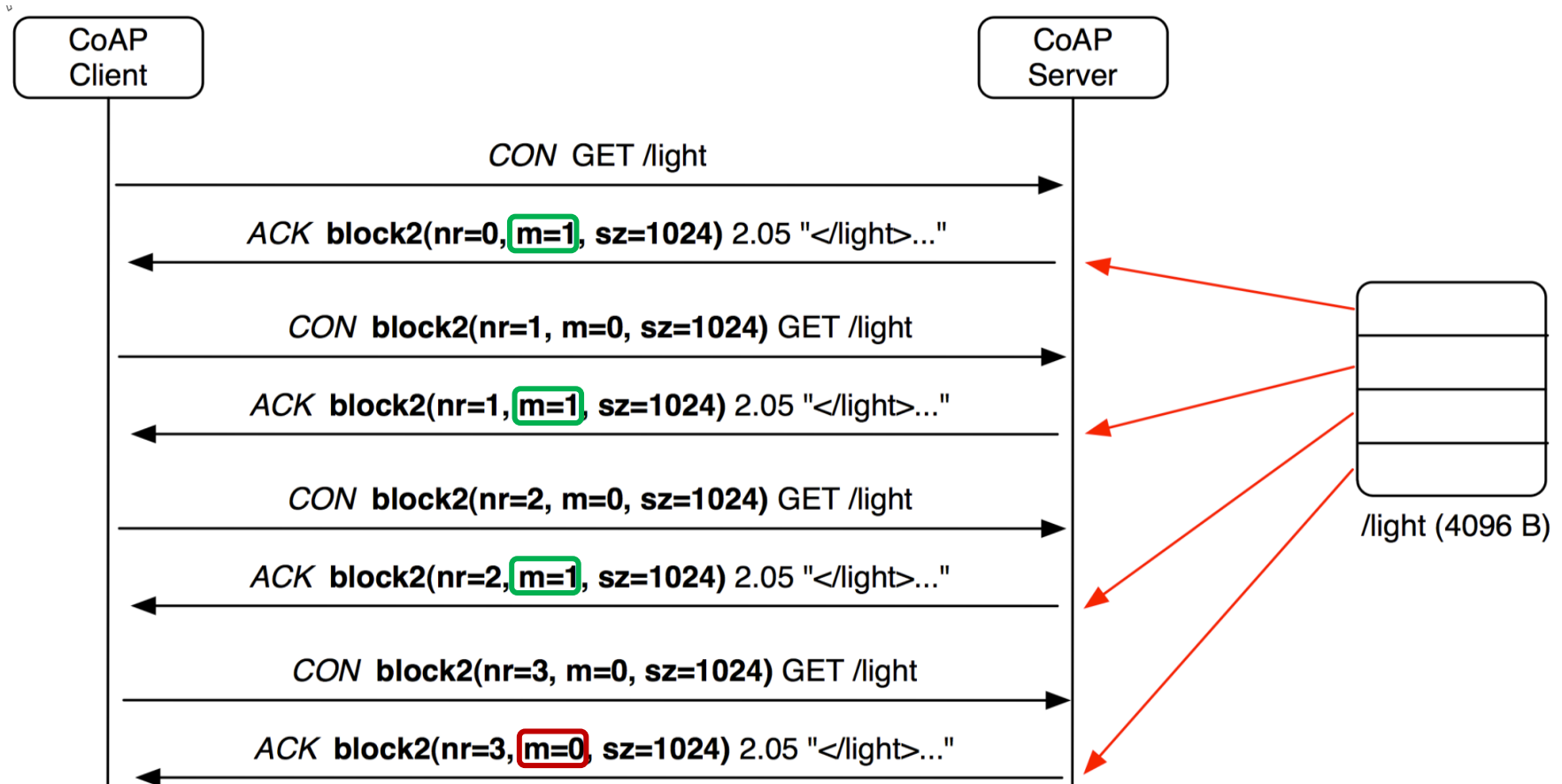
CoAP – Proxying & Caching



CoAP – Observe



CoAP – Transferencia Bloque



CoAP – Descubrimiento de Recursos

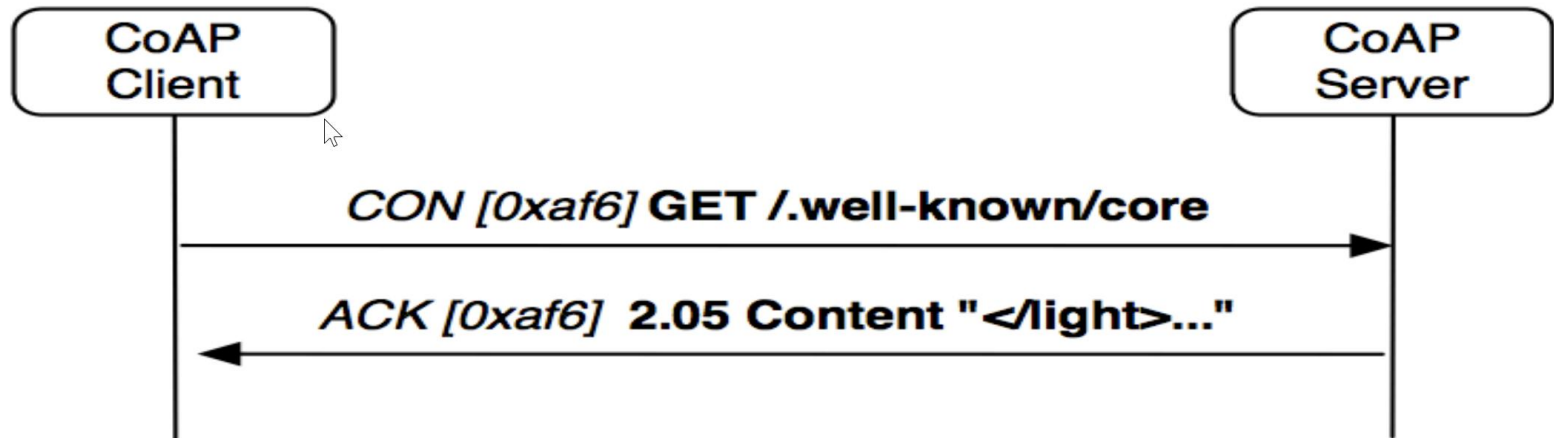
- Descubrimiento de servicios
 - ¿Qué servicios están disponibles en primer lugar?
 - Objetivo: encontrar dirección IP, el puerto y el protocolo
 - P. ej. mediante DNS-SD cuando DNS está disponible
- **Descubrimiento de recursos**
 - ¿**Cuáles** son los **recursos web** que me interesan?
 - **Objetivo: encontrar URIs**
 - Mediante **enlaces web** o alguna interfaz **REST**
 - El **formato de enlace CoRE** (Constrained RESTful Environments) está diseñado para descubrimiento de recursos

CoAP – Descubrimiento de recursos (II)

- RFC 6690 dedicado al descubrimiento de recursos M2M
 - Define una serialización de enlaces adecuada para M2M
 - Define un recurso conocido donde se almacenan enlaces
 - Habilita los parámetros de consulta para GET filtrados
 - Se puede utilizar con *unicast* o *multicast* (CoAP)
- Descubrimiento de recursos con RFC 6690
 - Descubrimiento de enlaces alojados en servidores CoAP (o HTTP)
 - GET */.well-known/core?optional_query_string*
 - Devuelve un formato de estilo de encabezado de enlace
 - URL, relación, tipo, interfaz, tipo de contenido, etc.

CoAP Resource Discovery (III)

- GET *./well-known/core*



```
</dev/bat>;obs;rt="ipso:dev-bat";ct="0",  
</dev/mdl>;rt="ipso:dev-mdl";ct="0",  
</dev/mfg>;rt="ipso:dev-mfg";ct="0",  
</pwr/0/rel>;obs;rt="ipso:pwr-rel";ct="0",  
</pwr/0/w>;obs;rt="ipso:pwr-w";ct="0",  
</sen/temp>;obs;rt="ucum:Cel";ct="0"
```

CoAP Resource Discovery (IV)

- RFC 6690 = Semántica sencilla para máquinas
 - [Constrained RESTful Environments \(CoRE\) Parameters \(iana.org\)](https://www.iana.org/assignments/core-parameters/core-parameters.xhtml)
 - Registro IANA para los parámetros *rt=* e *if=*
- Resource Type (*rt=*)
 - ¿En qué consiste este recurso y para qué sirve?
 - P. ej. modelo de dispositivo podría ser *rt="ipso.dev.mdl"*
- Interface Description (*if=*)
 - ¿Cómo accedo a este recurso?
 - P. ej., recurso de sensor accesible con GET *if="core.s"*
- Content Type (*ct=*)
 - ¿Cuál es el formato de datos de los *payloads* de recursos?
 - P. ej. *text/plain (0)*
- Observation Option (RFC 7641)
 - Indica que el destino de un enlace permite la observación

CoAP – Taller

- Instala *libcoap* & ejemplos
 - `sudo apt install libcoap3-bin`
- Instala el navegador Californium
 - Descarga el [JAR de Californium](#)
 - Instala JavaFX
 - `sudo apt install openjfx`
- Levanta el servidor CoAP
 - `coap-server-openssl -v 9 -N -k 123456789 -g FF02::FD`
 - Consulta la página del manual para comprender las opciones
- Lanza el navegador Californium
 - `java --module-path /usr/share/openjfx/lib --add-modules javafx.controls,javafx.fxml -jar cf-browser-3.6.0.jar`
 - Agrega `GDK_SCALE=2` al principio de la línea anterior en el caso de una pantalla 4K
- Lanza *wireshark* (como *root*)

CoAP – Taller (II)

- DISCOVERY en *coap://localhost:5683/*
 - Examina los recursos y sus características
- Ejecuta los siguientes comandos. Examina la captura de *wireshark* y explica los intercambios de mensajes
 - GET */time* CON
 - GET */time* NON
 - GET */async* CON (*respuesta asíncrona*)
 - GET */async* NON
 - OBSERVE */time* CON
 - GET */example_data* (*transmisión bloque*)
 - GET *coap://[ff02::fd]/time* (*solicitud multicast*)

CoAP – Taller (III)

- Ejecuta los siguientes comandos. Examina el *log* del servidor CoAP y la captura de *wireshark* y explica los intercambios de mensajes
 - Copia los datos (1500B) resultantes de un GET */example_data* y realiza un PUT */example_data* con esos datos
 - Repite lo mismo pero realizando un PUT de esos datos copiados dos veces (concatenados, 3000B)
- DESCUBRIMIENTO coap://coap.me
 - Examina los recursos y sus características
- Ejecuta los siguientes comandos. Examina el *log* del servidor CoAP y la captura de *wireshark* y explica los intercambios de mensajes
 - GET */hello*
 - GET */large*
 - POST */large_create* (con los datos obtenidos antes)
 - PUT */large_update* (con los datos obtenidos antes)